

Ereditarietà

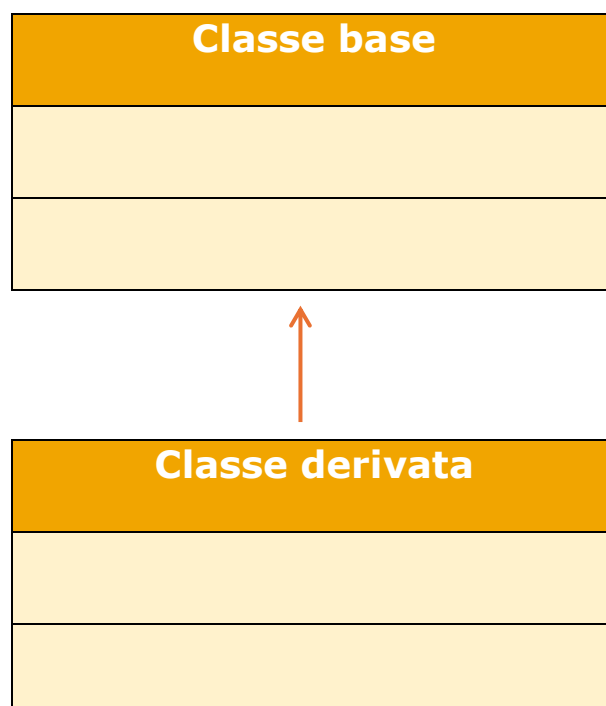
Quando sviluppiamo una applicazione o un sistema ad oggetti, dobbiamo identificare le classi che sono necessarie per risolvere il problema, con le quali poi creeremo gli oggetti che ci servono.

Spesso non è necessario partire da zero, ovvero creare delle classi ex-novo, ma è possibile prendere una classe già esistente ed estenderla per adattarle alle nuove necessità.

Ereditarietà

È un concetto fondamentale della programmazione ad oggetti che ci consente di creare nuove classi a partire da una classe già esistente, detta **classe base**. La nuova classe eredita tutti gli attributi ed i metodi della classe base e può essere estesa/arricchita con ulteriori attributi e/o metodi. La nuova classe che otteniamo viene detta **classe derivata**.

La **derivazione** di una classe in un diagramma UML delle classi viene rappresentata da una freccia che unisce i due rettangoli rappresentanti le classi, partendo dalla classe derivata e raggiungendo la classe base.



Il concetto di ereditarietà è ispirato dall'omonimo fenomeno che accade e si studia nella realtà, in natura: se ad esempio consideriamo i mammiferi, sappiamo che esistono diversi animali che appartengono a questa categoria (cane, gatto, uomo, balena...); ciascuna di queste specie di animali eredita caratteristiche comuni a tutti i mammiferi, come ad esempio il sangue caldo, sebbene ciascuna di esse possieda caratteristiche uniche (ad esempio l'uomo è l'unico mammifero in grado di parlare).

Sottoclasse

La classe che deriva da un'altra tramite il meccanismo dell'ereditarietà.

Superclasse (o sopraclasse)

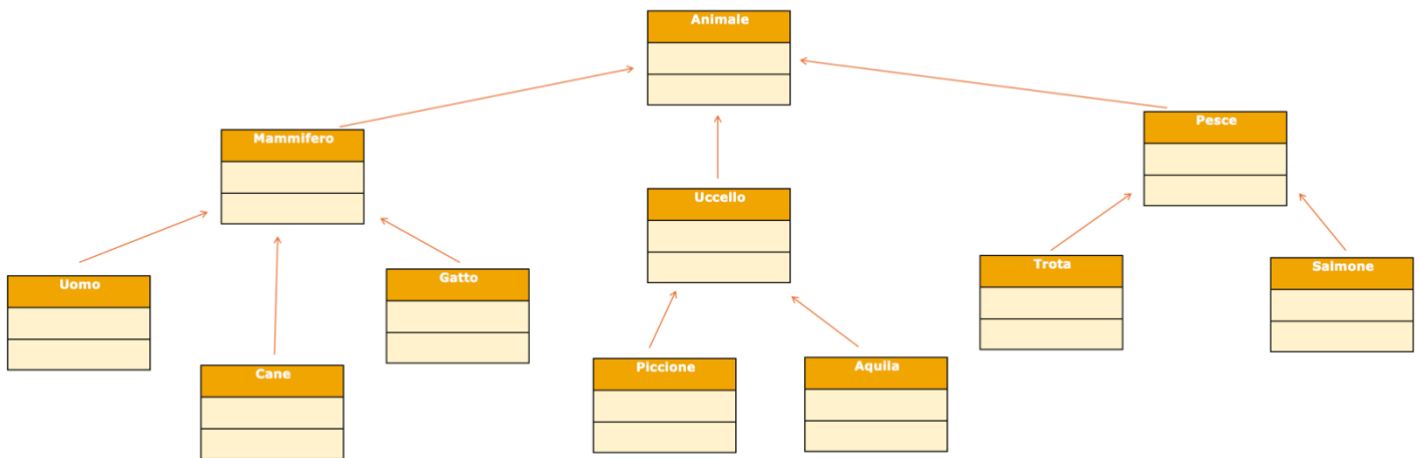
La classe che genera una sottoclasse.

Si viene quindi a definire una vera e propria **gerarchia di classi**. Questa si basa sul concetto di **specializzazione**: le classi più in alto nella gerarchia sono le più generali, mentre man mano che si scende troviamo le classi più specializzate.

La gerarchia può essere rappresentata graficamente tramite un **grafo orientato**, nel quale i nodi sono i diagrammi delle classi e gli archi sono rappresentati dalle frecce che esprimono una relazione di ereditarietà tra le classi (la direzione è dalla sottoclasse alla superclasse).

Facciamo un esempio: consideriamo la classe Animale. Questa possiamo intuitivamente associarla ad una superclasse, dalla quale derivano le sottoclassi Mammifero, Uccello e Pesce. A loro volta, dalla classe Mammifero derivano Uomo, Gatto e Cane, dalla classe Uccello derivano Piccione e Aquila ed infine dalla classe Pesce derivano Trota e Salmone.

Possiamo rappresentare questa gerarchia (assolutamente non esaustiva) tramite il seguente grafo:



Una sottoclasse riceve in eredità tutti gli attributi ed i metodi della superclasse; questo evita di dover riscrivere quegli elementi che sono comuni nelle sottoclassi, e ci consente di definire solamente le differenze che vengono in esse introdotte.

Una sottoclasse si differenzia dalla propria superclasse in due modi:

1. per **estensione**: la sottoclasse aggiunge attributi e/o metodi che non erano presenti nella superclasse e che si aggiungono a quelli ereditati;
2. per **ridefinizione**: la sottoclasse può ridefinire i metodi ereditati dalla superclasse, fornendo una implementazione diversa di uno stesso metodo, mantenendone invariato il nome (**overriding** del metodo). Quando un oggetto appartenente alla sottoclasse richiamerà tale metodo, non verrà eseguito quello della superclasse, ma quello ridefinito nella sottoclasse.

Polimorfismo

È un altro principio fondamentale della programmazione ad oggetti e rappresenta la capacità di un oggetto di poter assumere "molte forme". Più precisamente, consente di trattare oggetti di classi diverse in modo uniforme attraverso una interfaccia comune.

Ereditarietà singola e multipla

Possiamo avere due tipi di ereditarietà:

1. **singola**: una sottoclasse deriva solamente da una superclasse (questo non vieta alla superclasse di derivare a sua volta da più classi);
2. **multipla**: una sottoclasse deriva da due (o più) superclassi, andando quindi ad ereditare tutti i metodi e gli attributi di ogni superclasse.

Usando i diagrammi delle classi, l'ereditarietà multipla si indica con due (o più) frecce che dalla sottoclasse si dirigono verso le superclassi.

L'ereditarietà multipla è usata molto meno frequentemente rispetto a quella singola; inoltre, non è supportata da tutti i linguaggi di programmazione ad oggetti.